Industrial University of Ho Chi Minh City

Spring Security

Spring Security

- Spring Security is a framework that focuses on providing both authentication and authorization (access control) to java web application and SOAP/REST web services.
- Spring framework supports integration with many technologies
  - HTTP basic authentication
  - LDAP
  - OpenID providers
  - JAAS API
  - And customized authentication system (by yourself)

Spring Security

- **Concepts**
- Principal
  - User that performs the action
- Authentication
  - Confirming truth of credentials
- Authorization
  - Define access policy for principal
- GrantedAuthority
  - Application permission granted to a principal

Spring Security

- **Components**
- SecurityContext
  - Hold the authentication and other security information
- SecurityContextHolder
  - Provides access to SecurityContext
- AuthenticationManager
  - Controller in the authentication process
- AuthenticationProvider
  - Interface that maps to a data store which stores your user data.

Spring Security

- **Objects and interfaces involved in user authentication**
- Authentication Object
  - Object is created upon authentication, which holds the login credentials.
- UserDetails
  - Data object which contains the user credentials, but also the Roles of the user.
- UserDetailsService
  - Collects the user credentials, authorities(roles) and

Spring Security

`HttpSecurity` object in Spring Security is a central component for configuring web-based security for specific HTTP requests within a Spring application.

- **- Request-based Security:** allowing fine-grained control over authorization and authentication for different parts of an application.

- Default

Application: `HttpSecurity` configurations apply to all requests, but this can be restricted using methods like `requestMatcher(RequestMatcher)` to target specific URLs or patterns.

Spring Security

- **Fluent API:** `HttpSecurity` offers a chainable, fluent API for configuring various security features, including:

•**Authorization:** Defining access rules for different URLs or roles (`authorizeHttpRequests()`, `hasRole()`, `hasAnyRole()`, `permitAll()`).

•**Authentication:** Configuring authentication methods (e.g., form login, basic authentication, OAuth2).

•**Session Management:** Managing session creation, invalidation,

Exception Handling: Customizing how security-related exceptions are handled

Spring Security

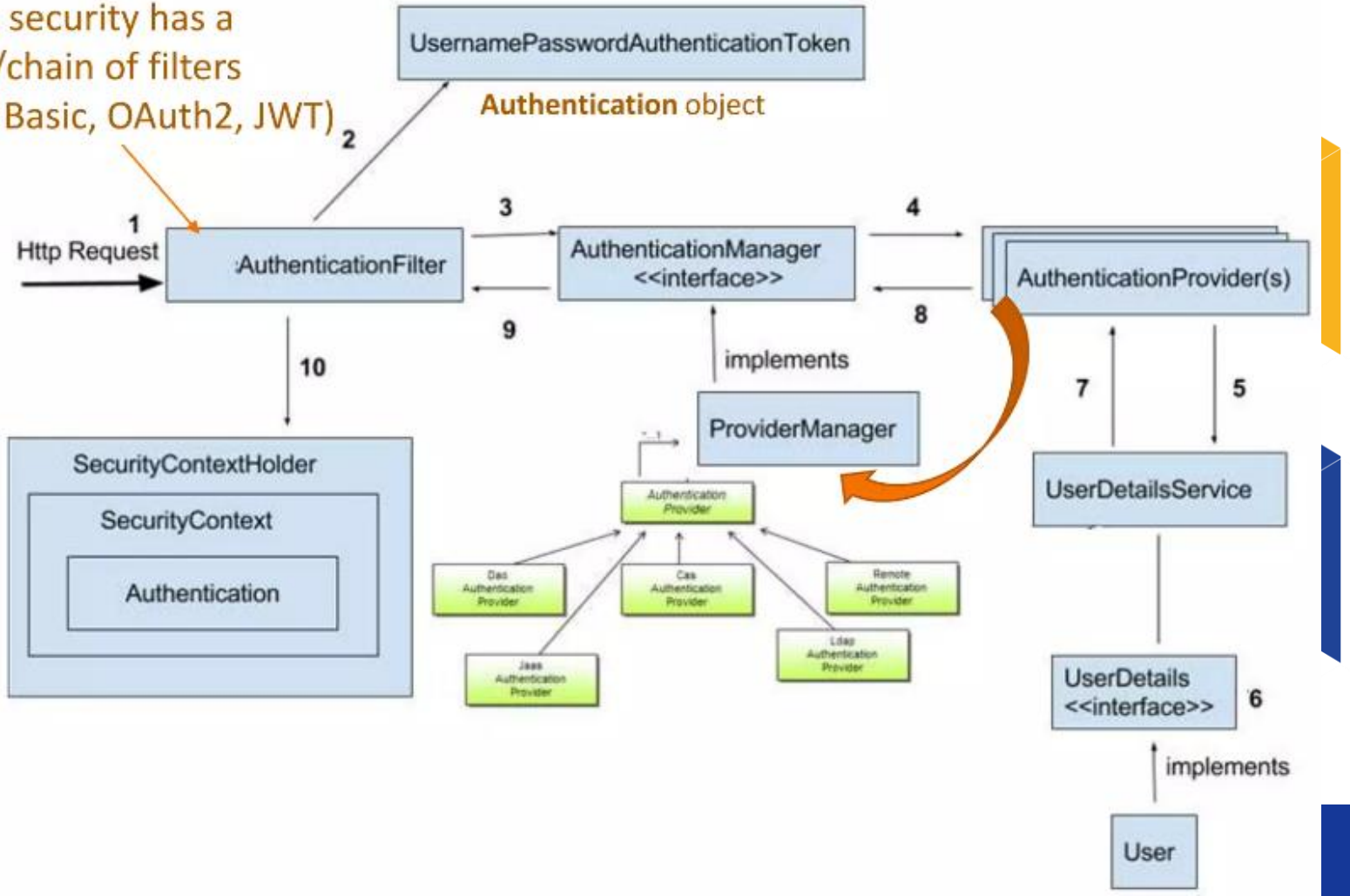
Integration

with SecurityFilterChain: HttpSecurity instances are typically configured within a SecurityFilterChain bean, which defines a chain of security filters applied to incoming requests

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Processing

- Spring security has a series/chain of filters (HTTP Basic, OAuth2, JWT)



Permision Methods

- resources for protecting:
  - permitAll()
  - denyAll()
  - hasAnyRole()
  - hasRole()
  - authenticated()

Spring Security

- Providing credentials ways:
  - In memory: credentials stored directly in the app (e.g., for testing)
  - Using JDBC: credentials fetched from a relational database.
  - Using OAuth2AuthorizeRequest: external authorization (e.g., Google, GitHub login).
  - Using LDAP: redentials validated against an LDAP directory (common in enterprises).
- In this study, **in memory** and using **JDBC credentials** are used.

In memory credential

Using BCryptPasswordEncoder for encoding the

```
@Bean  new *
public BCryptPasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

```
@Bean
public UserDetailsService userDetailsService() {
    UserDetails user = User.withDefaultPasswordEncoder()
            .username("admin")
            .password("123")
            .roles("ADMIN")
            .build();
    return new InMemoryUserDetailsManager(user);
}
```

JDBC: credentials

Using JDBC credentials (1)

```
@Autowired
DataSource dataSource;

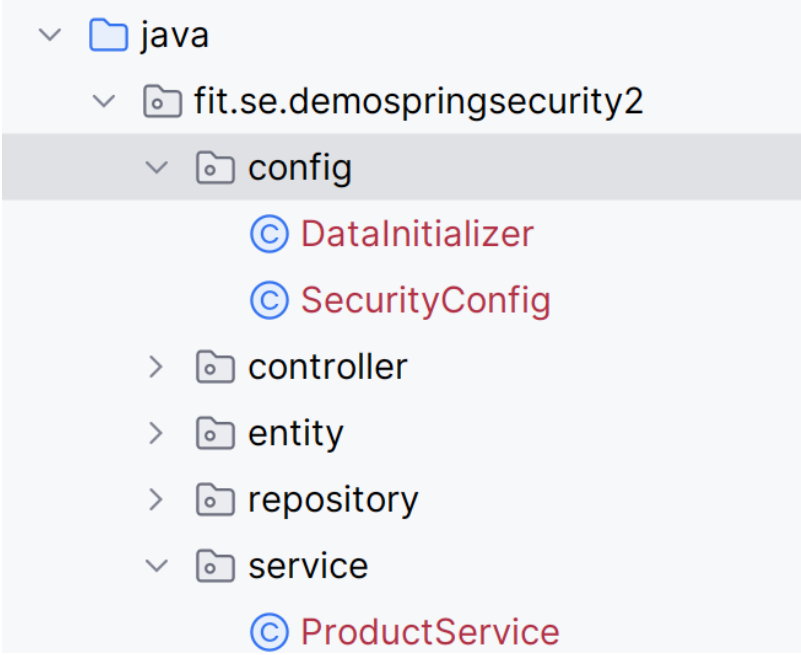
@Bean
public UserDetailsService users() {
    return new JdbcUserDetailsManager(dataSource);
}
```

- Security form

Login Forms

- The logging forms:
  - From web browser (system)
  - From spring security
  - Custom (specify the login form by yourself)

Demo


java
fit.se.demospringsecurity2
config
DataInitializer
SecurityConfig
controller
entity
repository
service
ProductService

Demo

- Steps:
  1. Add dependencies
  2. Create **config** package in project

  → Create **SercurityConfig** class and add annotation

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {
```

Demo

3. Create a bean with return type SecurityFilterChain and a parameter with type HttpSecurity with in method filter

```
.authorizeHttpRequests(auth -> auth
      .requestMatchers(" URIs ").permitAll()
      .requestMatchers("URIs").hasAnyRole("CUSTOMER", "ADMIN")
      .requestMatchers("URIs").hasRole("ADMIN")
      .anyRequest().authenticated()
      )
      .formLogin(form -> form
         .defaultSuccessUrl("/product")
         .permitAll()
          )
       .logout(logout -> logout
           .logoutUrl("/logout")
            .logoutSuccessUrl("/login?logout=true")
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID")
            .permitAll()
                );
```

Demo

4. Controller add annotation

@PreAuthorize("hasRole('ADMIN')") for method has only authorize

```
@PreAuthorize("hasRole('ADMIN')")  new *
@GetMapping("/add")
public String showAddForm(Model model) {
    model.addAttribute( attributeName: "product", new Product());
    model.addAttribute( attributeName: "categories", categoryService.findAll
    return "product/productform";
}
@PreAuthorize("hasRole('ADMIN')")  new *
@PostMapping("/save")
public String saveProduct(@ModelAttribute Product product,
                          @RequestParam("fileImage") MultipartFile file
                         ) throws IOException {

    if (!file.isEmpty()) {
        String fileName = file.getOriginalFilename();
```

- 5. View: check for ROLE for each action

```
<div sec:authorize="hasRole('ADMIN')">
    <a th:href="@{/products/add}" class="btn btn-primary">
        <i class="fas fa-plus"></i> Add New Product
    </a>
</div>
```

Spring Security

- OAuth2 (Single sign-On)
- JSON Web Token (JWT)
- OpenID Connect
- …

- Introduction

Cross-Site Request Forgery (CSRF)

- Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated.
- With a little help of social engineering (such as sending a link via email or chat), an attacker may trick the users of a web application into executing actions of the attacker's choosing.
- If the victim is a normal user, a successful CSRF attack can force the user to perform state changing requests like transferring funds, changing their email address, and so forth.
- If the victim is an administrative account, CSRF can compromise the entire web application.
- There are multiple forms of CSRF attack. Two simple CSRF attacks that most common happen:
  - Using GET method
  - Using POST method

CSRF attacks

Using get method

- Let's consider the following GET request used by a logged-in user to transfer money to a specific bank account 1234

```
GET http://bank.com/transfer?accountNo=1234&amount=100
```

- If the attacker wants to transfer money from a victim's account to his own account instead — 5678 — he needs to make the victim trigger the request:

```
GET http://bank.com/transfer?accountNo=5678&amount=1000
```

- There are multiple ways to make that happen:
  - Link – The attacker can convince the victim to click on this link, for example, to execute the transfer:

```
<a href="http://bank.com/transfer?accountNo=5678&amount=1000">
   Show Kittens Pictures
</a>
```

  - Image – The attacker may use an <img/> tag with the target URL as the image source. In other words, the click isn't even necessary. The request

```
<img src="http://bank.com/transfer?accountNo=5678&amount=1000"/>
```

CSRF attacks

Using POST method

- Suppose the main request needs to be a POST request:

```
POST http://bank.com/transfer
accountNo=1234&amount=100
```

- In this case, the attacker needs to have the victim run a similar request:

```
POST http://bank.com/transfer
accountNo=5678&amount=1000
```

- N
- T

ll work in this case:

```
<form action="http://bank.com/transfer" method="POST">
   <input type="hidden" name="accountNo" value="5678"/>
   <input type="hidden" name="amount" value="1000"/>
   <input type="submit" value="Show Kittens Pictures"/>
</form>
```

The form can be submitted automatically using JavaScript:

```
<body onload="document.forms[0].submit()">
<form>
  ...
</form>
```

Spring MVC web with CSRF

Prevent CRSF attack

- To protect MVC applications, Spring adds a CSRF token to each generated view.
- This token must be submitted to the server on every HTTP request that modifies state (PATCH, POST, PUT and DELETE — not GET).
- This protects our application against CSRF attacks since an attacker can't get this token from their own page.
- Starting from Spring Security 4.x, the CSRF protection is enabled by default.
- If we need to, we can disable this configuration but not recommended

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
   http
        .csrf().disable();
   return http.build();
}
```

Spring MVC web with CSRF

Client prevent CSRF attack configuration

- We need to include the CSRF token in our requests. The _csrf attribute contains the following information:
  - *token* – the CSRF token value
  - *parameterName* – name of the HTML form parameter, which must include the token value
  - *headerName* – name of the HTTP header, which must include the token valu

```
<input type="hidden" name="${_csrf.parameterName}" value="${_csrf.token}"/>
```

- In case of using HTML forms, the *headerName* and *token* values should be added HTTP header.

```
//1. include the token value and the header name in meta tags
<meta name="_csrf" content="${_csrf.token}"/>
<meta name="_csrf_header" content="${_csrf.headerName}"/
```

- the *headerName* and *token* values

```
//2. Then let's retrieve the meta tag values with JQuery:
var token = $("meta[name='_csrf']").attr("content");
var header = $("meta[name='_csrf_header']").attr("content");
```

```
// 3. Finally, let's use these values to set our XHR header:
$(document).ajaxSend(function (e, xhr, options) {
   xhr.setRequestHeader(header, token);
   });
```

Control the Session with Spring Security

Control the Session with Spring Security

- https://www.baeldung.com/spring-security-session

